

Overview

Part 11 —  The LLM Model Landscape · 11-llm-models/README.md

Module 11 — The LLM Model Landscape

Where you are: Stop 11 of 16. **Time:** ~18 hours · **Prereq:** Module 04 [04-llm] — non-negotiable.

Module 04 taught you how an LLM is built and how to run one well. This module teaches you how to **choose one** — and how to keep choosing correctly as the landscape reshuffles under you, which it does on roughly a monthly cadence. Every fact in here about a specific model has a shelf life measured in weeks; every framework in here for choosing a model has a shelf life measured in years. Learn the framework. Verify the facts before you rely on them.

Learning objectives

1. Classify any model into frontier, mid-tier, or small/fast — and explain why every major provider converges on the same three-tier structure (plus a reasoning variant) despite competing with each other.
2. Choose proprietary vs. open-weights for a given task and defend it on data residency, customization depth, cost-at-scale, support, and licensing — not vibes.
3. Build a defensible model recommendation from a **task profile** (accuracy bar, latency budget, cost budget, compliance constraints) scored against **your own eval set**, never against a public leaderboard alone.
4. Explain what MMLU, GPQA, SWE-bench, and Arena-style Elo actually measure, and name the failure mode each is most often misused to hide (contamination, overfitting, verbosity bias, non-representative tasks).
5. Match context-window and multimodal requirements to the actual task instead of defaulting to the biggest advertised number, using "usable context" as distinct from "advertised context."
6. Design an application so it is not locked to one model or one provider, and produce a runbook for swapping a production model without a silent quality regression.
7. Treat model choice as an ongoing maintenance obligation: pin versions deliberately, track deprecation timelines, and catch a "latest" alias changing under you with your eval suite.

Suggested path

#	Do this	File	Time
1	The landscape: tiers, open vs. proprietary, benchmarks	notes/01-the-model-landscape.md	4h
2	Selecting, migrating, and maintaining a model choice	notes/02-selecting-and-migrating-models.md	4h
3	Score candidates against a real task profile	code/model_selection_scorer.py	2h
4	See a leaderboard number lie to you	code/benchmark_literacy_demo.py	1h
5	Slides	slides/	1h
6	Lab	lab/EXERCISES.md	4h
7	Papers	papers/PAPERS.md	2h

The 16 terms you must own

frontier tier · open-weights · permissive vs. restrictive license · benchmark contamination · Arena Elo / verbosity bias · usable context vs advertised context · model gateway · prompt portability · tool-calling schema drift · model pinning · "latest" alias risk · deprecation window · eval regression suite · self-hosting break-even · domain-specialized model · Goodhart's law (leaderboards)

Exit check

Given a real (or realistic) use case, you can produce a one-page model-selection memo: the task profile you defined first, the candidates you scored against it with reasoning shown (not a bare number), the model you'd ship, the version you'd pin, and the migration/rollback plan for the day the provider deprecates it or silently updates the alias you didn't pin. Run both scripts in `code/` to check your reasoning against a transparent, editable model.